# MANUAL TÉCNICO DE INGENIERÍA

## Aplicativo Web para el Manejo de Finanzas Personales

**Desarrollador: Juan Esteban Sanchez**

**Arquitectura: Asíncrona basada en FastAPI y SQLite**

**Fecha: 01 de April, 2026**

# Índice Detallado

Capítulo 1: Resumen Ejecutivo del Proyecto

Capítulo 2: Arquitectura del Sistema (Frontend/Backend)

Capítulo 3: Stack Tecnológico Detallado

Capítulo 4: Frontend: Motor de Estilos Glassmorphism

Capítulo 5: Backend: FastAPI y Manejo de Concurrencia

Capítulo 6: Base de Datos: Modelo Entidad-Relación y ACID

Capítulo 7: Seguridad Aplicativa: JWT, MFA y Criptografía

Capítulo 8: Integración con Servicios de Google Firebase

Capítulo 9: Infraestructura de Despliegue: Vercel Cloud

Capítulo 10: Documentación de la API (RESTful Endpoints)

Capítulo 11: Lógica de Negocio: Algoritmos Contables

Capítulo 12: Módulo de Educación Financiera y Didáctica

Capítulo 13: Escalabilidad y Roadmap de Mejoras

Capítulo 14: Gestión de Calidad y Pruebas Unitarias

## Capítulo 15: Conclusiones Arquitectónicas

## Capítulo 16: Glosario de Ingeniería de Software

# Capítulo 1: Ingeniería del Sistema Financiero

El Aplicativo Web para el Manejo de Finanzas Personales ha sido desarrollado como una solución integral para la gestión de activos, pasivos y flujo de caja individual. A diferencia de sistemas genéricos de registro, esta plataforma implementa lógica contable rigurosa y un motor de visualización de datos basado en ApexCharts para proporcionar una visión 360 de la salud financiera del usuario.

La arquitectura se diseñó bajo el principio de 'Seguridad por Diseño', utilizando FastAPI como middleware de alto rendimiento que garantiza que cada movimiento bancario, deuda o inversión sea procesado con integridad ACID en una base de datos SQLite persistente.

El objetivo técnico principal es la reducción de la fricción en el registro, utilizando una interfaz asíncrona que elimina las recargas de página y permite al usuario gestionar sus finanzas con la velocidad de una aplicación nativa, pero con la accesibilidad de la web.

# Capítulo 2: Arquitectura del Sistema

## Visión General Cliente-Servidor

La arquitectura se basa en el modelo cliente-servidor enriquecido. El backend actúa como una API RESTful que sirve datos en formato JSON, mientras que el frontend actúa como una Single Page Application (SPA) que gestiona el estado de manera asíncrona.

Figura 1: Representación visual del Dashboard Central y su arquitectura de componentes.

## Diagrama de Capas (Lógico)

1. Capa de Presentación: HTML5 semántico, CSS3 con variables

personalizadas y JavaScript ES6+.

2. Capa de Lógica (Router): FastAPI orquestando los endpoints y validando esquemas.

3. Capa de Servicios: Integración con Firebase para autenticación y envío de correos SMTP.

4. Capa de Datos: SQLite operando con transacciones ACID para asegurar la integridad contable.

La comunicación entre capas se realiza mediante protocolos HTTP/S seguros, utilizando cabeceras de autorización Bearer Tokens para cada transacción.

La comunicación entre capas se realiza mediante protocolos HTTP/S seguros, utilizando cabeceras de autorización Bearer Tokens para cada transacción.

La comunicación entre capas se realiza mediante protocolos HTTP/S seguros, utilizando cabeceras de autorización Bearer Tokens para cada transacción.

La comunicación entre capas se realiza mediante protocolos HTTP/S seguros, utilizando cabeceras de autorización Bearer Tokens para cada transacción.

La comunicación entre capas se realiza mediante protocolos HTTP/S seguros, utilizando cabeceras de autorización Bearer Tokens para cada transacción.

# Capítulo 6: Base de Datos

## Modelo Entidad-Relación

La base de datos SQLite ha sido normalizada a la Tercera Forma Normal (3NF). Esto garantiza que no existan redundancias indeseadas y que cada unidad de información dependa exclusivamente de una llave primaria única.

```
CREATE TABLE IF NOT EXISTS movements (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_email TEXT,
    type TEXT, -- 'income' or 'expense'
    concept TEXT,
    amount REAL,
    date TEXT,
    category TEXT
);
```

Otras tablas críticas incluyen 'debts', 'payments' y 'user_profiles', todas vinculadas mediante el correo electrónico del usuario como llave foránea virtual indexada para búsquedas ultrarrápidas.

# Capítulo 7: Seguridad y MFA

La seguridad es el pilar de este aplicativo financiero. Se ha implementado un sistema de autenticación de doble factor (MFA) personalizado.

1. Autenticación Inicial: Gestionada por Firebase Identity Platform, asegurando que las contraseñas nunca toquen nuestros servidores directamente.

2. Desafío de Seguridad (OTP): El backend genera un código de 6 dígitos único vinculado al correo del usuario. Este código tiene una validez temporal de 10 minutos.

3. Verificación de Transacciones: Cada acceso crítico requiere la validación exitosa del código OTP, mitigando riesgos de acceso no autorizado incluso si las credenciales primarias son comprometidas.

# Capítulo 11: Algoritmos de Cálculo Financiero

## Control de Pasivos y Deudas

El sistema calcula automáticamente el impacto de las deudas en el flujo de caja. Utiliza algoritmos de amortización para proyectar pagos y sugerir estrategias de liquidación prioritarias (método avalancha).

El motor de presupuestos aplica la regla 50/30/20 de manera dinámica basándose en los ingresos netos registrados durante el ciclo contable mensual.

# Capítulo 16: Glosario de Ingeniería

**API REST:**

Interfaz de programación de aplicaciones que utiliza peticiones HTTP.

**ACID:**

Atomicidad, Consistencia, Aislamiento y Durabilidad en bases de datos.

**FastAPI:**

Framework moderno para construir APIs con Python basado en tipos.

**JWT:**

JSON Web Token utilizado para la transmisión segura de información.

**MFA:**

Multi-Factor Authentication, sistema de seguridad de varios pasos.

**Glassmorphism:**

Tendencia de diseño basada en la apariencia de vidrio esmerilado.